

CSCE 2014 - Laboratory Assignment 10

The purpose of this lab is to evaluate the performance of two well known sorting algorithms: bubble sort and selection sort. We will do this by calculating the cpu times to sort different types of input data (random, mostly-sorted, and sorted) using these two sorting algorithms. As we will see there can be some dramatic differences in performance.

1. Introduction to Sorting

The purpose of sorting is to put a sequence of data records in ascending or descending order based on a sorting key. For example, to sort student records based on last name, or sort baseball player records based on batting average. The problem of sorting data has been around a long, long time. See the Wikipedia entries for "Hollerith card" and "tabulating machine" for more information.

There have been dozens of different sorting algorithms invented over the years, each with different advantages and disadvantages. In the next two labs, we will be studying six classic sorting techniques: bubble sort, insertion sort, selection sort, quick sort, merge sort, and bucket sort. These are described in more detail in your text book, and also on the "sorting" Wikipedia page.

2. Generating Unsorted Data

There are two ways to generate unsorted data for sorting experiments. The first way is to use a random number generator and create N values within a range [L..H]. This is like rolling dice over and over again. The data values in this case will be in random order.

The second method starts with sorted data and randomly swaps pairs of values in the array. If we do lots of swaps, the array will eventually look like it is in random order. If we only do a small number of swaps, we will have data that is mostly in sorted order.

These two approaches are used in the code below to initialize an array of integers.

```
//-----  
// Initialize data array with random values  
//-----  
void create_random_data(int data[], int count, int range)  
{  
    // Put specified count of random numbers into data array  
    for (int index = 0; index < count; index++)  
        data[index] = rand() % range;  
}  
  
//-----  
// Initialize data array with mostly sorted values  
//-----  
void create_mostly_sorted_data(int data[], int count, int swaps)  
{  
    // Put sorted data values into array
```

```

    for (int index = 0; index < count; index++)
        data[index] = index;

    // Shuffle data by swapping random pairs of values
    for (int index = 0; index < swaps; index++)
    {
        int pos1 = rand() % count;
        int pos2 = rand() % count;
        int temp = data[pos1];
        data[pos1] = data[pos2];
        data[pos2] = temp;
    }
}

//-----
// Write data array to output file
//-----
void write_data(string name, int data[], int count)
{
    // Open output file
    ofstream dout;
    dout.open(name.c_str());
    if (dout.fail())
        cout << "Error: could not open output file\n";

    // Write the data
    dout << count;
    for (int i = 0; i < count; i++)
    {
        if (i % 20 == 0)
            dout << endl;
        dout << data[i] << " ";
    }

    // Close the file
    dout.close();
}

```

Create a new program in your IDE and cut and paste the functions "create_random_data", "create_mostly_sorted_data" and "write_data" above into your program. Now create a main program that declares an array of 100 integers, and calls "create_random_data" and "write_data" to create a data file "random.txt" with random values between 0 and 99. If you look at this file, it should look almost as *unsorted* as possible.

Now extend your main program to call the "create_mostly_sorted_data" function on an array of 100 integers to create the file "mostly_sorted.txt". Try a few different values of the "swaps" parameter to see what happens and choose a value that creates data that looks *mostly sorted* (some sequences of values are in the correct order, but others are out of order).

Cut and paste your main program below:

```
int main() {
    const int MAX_SIZE = 100;
    int data[MAX_SIZE];

    create_random_data(data, MAX_SIZE, 99);
    write_data("random.txt", data, MAX_SIZE);

    create_mostly_sorted_data(data, MAX_SIZE, 42);
    write_data("mostly_sorted.txt", data, MAX_SIZE);

    return 0;
}
```

Cut and paste one of your mostly sorted results below:

```
100
0 28 2 3 94 26 54 7 4 31 10 11 12 70 14 15 9 85 18 68
44 67 22 47 24 25 60 27 71 29 13 90 32 33 66 62 36 58 63 35
79 41 65 43 16 75 46 98 48 49 59 51 82 53 23 55 56 38 37 50
89 61 6 8 64 99 34 21 95 69 30 97 39 73 74 45 76 1 78 40
80 81 52 83 84 17 86 87 88 57 20 91 92 93 5 19 96 77 72 42
```

3. Comparing Bubble Sort and Selection Sort

Cut and paste the functions "bubble_sort" and "selection_sort" below into your program.

```
//-----
// Bubble sort algorithm
//-----
void bubble_sort(int data[], int count)
{
    int pass = 1;
    int exchange = 1;

    // Bubble largest value to the right N times
    while ((pass < count) && (exchange > 0))
    {
        // Scan unsorted part of data array
        exchange = 0;
        for (int index = 0; index < count - pass; index++)
        {
            // Swap two data values if out of order
            if (data[index] > data[index + 1])
            {
                int temp = data[index];
                data[index] = data[index + 1];
                data[index + 1] = temp;
                exchange++;
            }
        }
        pass++;
    }
}

//-----
```

```
// Selection sort algorithm
//-----
void selection_sort(int data[], int low, int high)
{
    // Put largest unsorted value at end of sorted list
    for (int last = high; last > low; last--)
    {
        // Find index of largest value in unsorted array
        int largest = low;
        for (int index = low + 1; index <= last; index++)
            if (data[index] > data[largest])
                largest = index;

        // Swap with last element in unsorted array
        int temp = data[last];
        data[last] = data[largest];
        data[largest] = temp;
    }
}
```

Now we are ready to do some experiments to compare the speed of these two sorting algorithms. Your next task is to modify your main program to calculate the CPU times for the following tasks:

- bubble_sort with N random data values between 0..N-1
- bubble_sort with N mostly sorted data values (swaps = N/10)
- bubble_sort with N sorted data values (swaps = 0)
- selection_sort with N random data values between 0..N-1
- selection_sort with N mostly sorted data values (swaps = N/10)
- selection_sort with N sorted data values (swaps = 0)

Use the code below to measure the CPU time. Calculate the run times for N=1000. If the CPU times are too small to measure, increase N as needed until you can get a CPU time greater than a few seconds. Stop your program if it uses more than a minute of CPU time and decrease N accordingly.

```
// Get start time
clock_t time1=clock();

// Some chunk of code

// Get end time
clock_t time2=clock();
double run_time = (time2-time1)/(double)CLOCKS_PER_SEC;
cout << "Run time: " << run_time << " seconds\n";
```

Cut and paste your modified main program below:

```

int main() {
    const int N = 70000;

    // Get start time
    clock_t time1 = clock();

    // bubble_sort with N random data values between 0..N-1
    int data[N];
    create_random_data(data, N, N);
    bubble_sort(data, N);

    // Get end time
    clock_t time2 = clock();
    double run_time = (time2 - time1) / (double)CLOCKS_PER_SEC;
    cout << "bubble_sort with N random data values between 0..N-1: " << run_time
        << " seconds\n";

    // bubble_sort with mostly sorted data (swaps = N/10)
    create_mostly_sorted_data(data, N, N / 10);
    clock_t time3 = clock();
    bubble_sort(data, N);
    clock_t time4 = clock();
    double run_time2 = (time4 - time3) / (double)CLOCKS_PER_SEC;
    cout << "bubble_sort with mostly sorted data (swaps = N/10): " << run_time2
        << " seconds\n";

    // bubble_sort with sorted data (swaps = 0)
    create_mostly_sorted_data(data, N, 0);
    clock_t time5 = clock();
    bubble_sort(data, N);
    clock_t time6 = clock();
    double run_time3 = (time6 - time5) / (double)CLOCKS_PER_SEC;
    cout << "bubble_sort with sorted data (swaps = 0): " << run_time3
        << " seconds\n";

    // selection_sort with random data
    create_random_data(data, N, N);
    clock_t time7 = clock();
    selection_sort(data, 0, N - 1);
    clock_t time8 = clock();
    double run_time4 = (time8 - time7) / (double)CLOCKS_PER_SEC;
    cout << "selection_sort with random data: " << run_time4 << " seconds\n";

    // selection_sort with mostly sorted data (swaps = N/10)
    create_mostly_sorted_data(data, N, N / 10);
    clock_t time9 = clock();
    selection_sort(data, 0, N - 1);
    clock_t time10 = clock();
    double run_time5 = (time10 - time9) / (double)CLOCKS_PER_SEC;
    cout << "selection_sort with mostly sorted data (swaps = N/10): " << run_time5
        << " seconds\n";

    // selection_sort with sorted data (swaps = 0)
    create_mostly_sorted_data(data, N, 0);
    clock_t time11 = clock();
    selection_sort(data, 0, N - 1);
    clock_t time12 = clock();
    double run_time6 = (time12 - time11) / (double)CLOCKS_PER_SEC;
    cout << "selection_sort with sorted data (swaps = 0): " << run_time6
        << " seconds\n";
    return 0;
}

```

Enter the results of your six timing experiments below:

```

bubble_sort with N random data values between 0..N-1: 9.871 seconds
bubble_sort with mostly sorted data (swaps = N/10): 4.65007 seconds
bubble_sort with sorted data (swaps = 0): 9.4e-05 seconds
selection_sort with random data: 3.186 seconds
selection_sort with mostly sorted data (swaps = N/10): 3.22167 seconds
selection_sort with sorted data (swaps = 0): 4.00708 seconds

```

4. Submit Work

When you have completed all of the steps to this lab:

- At the bottom of the page enter your name and UAID and click "Submit Lab".
- Print the resulting lab report web page to a PDF file.
- Login to Blackboard and go to the labs section.
- Upload your PDF file for grading and feedback.

Enter your FIRST name:

Enter your LAST name:

Enter your UAID number:

Submit Lab